

20 Matrices

Content

- Definition
- print row wise & col wise sum
- print diagonal & anti diagonal
- Diagonal traversal
- Transpose of a square matrix
- Rotate Matrix

Amreshwar

ANUBHAV RAMNANI

Arunava Basak

Bhavesb Pandey

Bhushan Ganesh Shelar

Deepshikha Arora

Farzana kauser

Iti

M S Haseeb Khan

Mahesh

Manohar A N

MOHAMMAD ALI

Nutan

Pranjul Kesharwani

Praveen Kumar

Shivam Shiv

Shivanand Patil

Tanya

Ved Verma

Rules

Q $\longrightarrow$ QT

A $\longrightarrow$ Private chat

Always active

2D Matrices Definition

An array of array is a 2D matrix.

A matrix of size $N \times M$ represents

$N \longrightarrow$ no. of rows

$M \longrightarrow$ no. of cols.

	0	1	2
0	0,0	0,1	0,2
1	1,0	1,1	1,2
2	2,0	2,1	2,2
3	3,0	3,1	3,2

$N = 4$

$M = 3$

4×3 matrix

$A = [[0] \times M \text{ for } _ \text{ in range}(N)]$

How to initialize $\longrightarrow$ `new int[N][M]`

Index of top right corner $0, M-1$

Index of bottom right corner $N-1, M-1$

	0	1	2	
0	(0,0)	(0,1)	(0,2)	← Column Index
1	(1,0)	(1,1)	(1,2)	
2	(2,0)	(2,1)	(2,2)	
↑ Row Index				

Q> Given a 2D matrix
Print the row wise sum

A [4][3]	0	1	2	output
0	1	2	3	6
1	4	5	6	15
2	7	8	9	24
3	10	11	12	33

N $\longrightarrow$ No. of rows

M $\longrightarrow$ No. of cols

```
for x  $\longrightarrow$  0 to N-1 {  $\longrightarrow$  N times
    sum = 0
    for c  $\longrightarrow$  0 to M-1 {  $\longrightarrow$  M times
        sum += A[x][c]
    }
    print(sum)
}
```

TC: $O(N*M)$
SC: $O(1)$

Q> Given a 2D matrix
Print the col wise sum

mat[4][3]

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12
	22	26	30

output

```
for c → 0 to M-1 {  
    sum = 0  
    for r → 0 to N-1 {  
        sum += A[r][c]  
    }  
    print (sum)  
}
```

TC: $O(N*M)$
SC: $O(1)$

Q> Given 2D matrix $\text{mat}[N][N]$ { square matrix }
 Print the diagonal and anti diagonal

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

anti diagonal

diagonal

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

output
 1 6 11 16

Observation

$r == c$
 for diagonal

start = (0, 0)

(1, 1)

(2, 2).....

```

void printDiagonal (int[][] mat) {
    r = 0    c = 0
    while (r < N && c < N) {
        print (mat[r][c] + " ")
        r += 1
        c += 1
    }
}
    
```

way 1

```

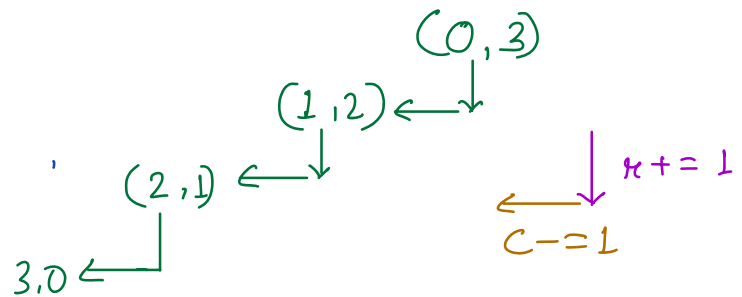
for r -> 0 to N-1 {
    print (mat[r][r] + " ")
}
    
```

way 2
 TC: $O(N)$

	0	1	2	(0, N-1)
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16

Output

4 7 10 13



Observation $\longrightarrow$ sum of $r+c = \text{constant} = N-1$

// start from top right cell

```

r = 0    c = N-1
while (r < N && c >= 0) {
    print (A[r][c])
    r += 1
    c -= 1
}

```

way 1

from observation

$r+c = N-1$

$c = N-1-r$

```

for r  $\longrightarrow$  0 to N-1
    c = N-1-r
    print (A[r][c])
}

```

TC: $O(N)$

SC: $O(1)$

Given a 2D matrix

Print all the anti diagonals from right to left.

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

mat[3][u]

output

1
2 5
3 6 9
4 7 10
8 11
12

No. of diagonals of matrix[N][M] = $M + N - 1$

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

// iterate over the first row

for $j \rightarrow 0$ to $M-1$ {

$r = 0$ $c = j$

while ($r < N$ && $c \geq 0$) {

print (A[r][c] + " ")

$r++$

$c--$

}

}
iterate
over first
row

```

    print("\n")
}
j 0 1 2 3 4
k
c

```

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12

```

for i → 1 to N-1 {
    k = i    c = M-1
    while (k < N && c >= 0) {
        print(A[k][c] + " ")
        k += 1
        c -= 1
    }
}
print("\n")

```

$i \neq 2$
 k
 c

Iterate over last col

NOTE: Every cell in the above matrix is iterated only once
 How many cells are there? $N * M$

$$TC: O(N * M * 1) = O(N * M)$$

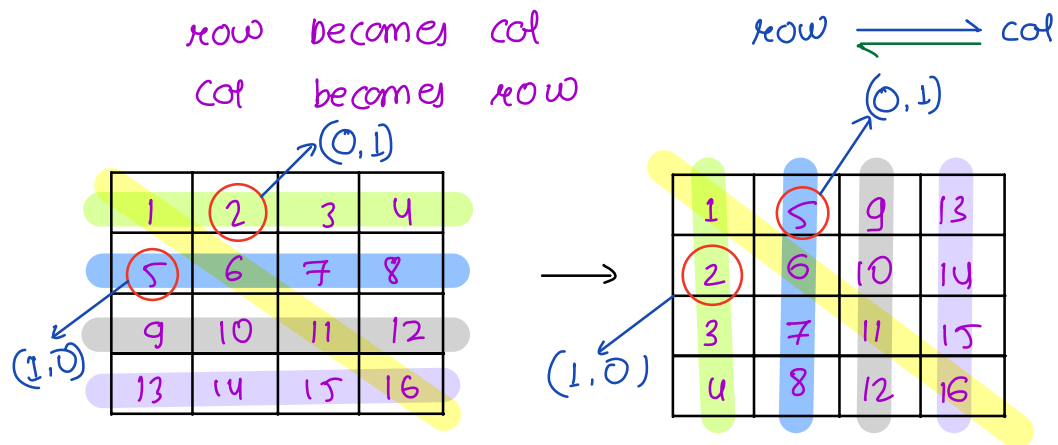
Break 22:18

Q> Given a 2D matrix $N \times N$ {square matrix}
Find the transpose

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

→

1	5	9	13
2	6	10	14
3	7	11	15
4	8	12	16



Let's try to swap $\rightarrow A[r][c]$ with $A[c][r]$

```

for r  $\rightarrow$  0 to N-1 {
  for c  $\rightarrow$  0 to N-1 {
    swap ( A , r , c ) // swap A[r][c] with A[c][r]
  }
}

```

Diagram illustrating the swap operation. The original matrix is:

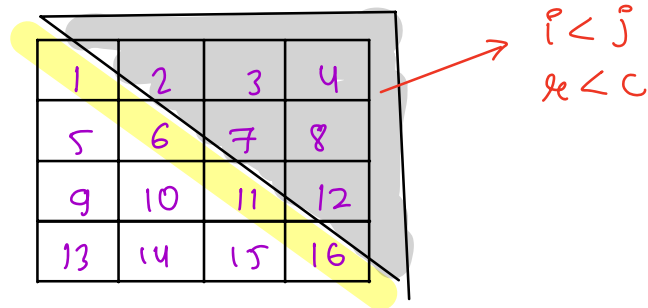
1	2
3	4

After swapping, the matrix becomes:

1	3
2	4

$\therefore$ we are swapping 1,0 and 0,1 two times

How to ensure we only swap once



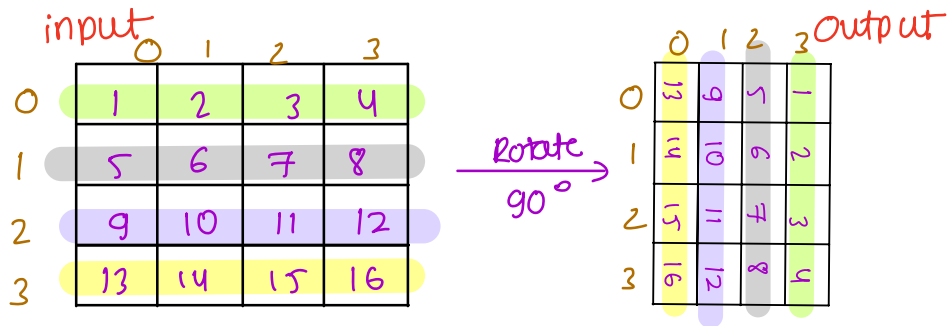
```
for x → 0 to N-1
  for c → x+1 to N-1 {
    swap (A, x, c)
  }
```

temporal

```
void swap (int[][] A, int x, int c) {
  temp = A[x][c]
  A[x][c] = A[c][x]
  A[c][x] = temp
}
```

x	c	iterations	
0	1 ... N-1	N-1	$\frac{N * (N-1)}{2}$ TC: $O(N^2)$
1	2 ... N-1	N-2	
2	3 ... N-1	N-3	
		...	
		1	

Given a 2D matrix $N \times N$ {square matrix}
 Rotate by 90° clockwise

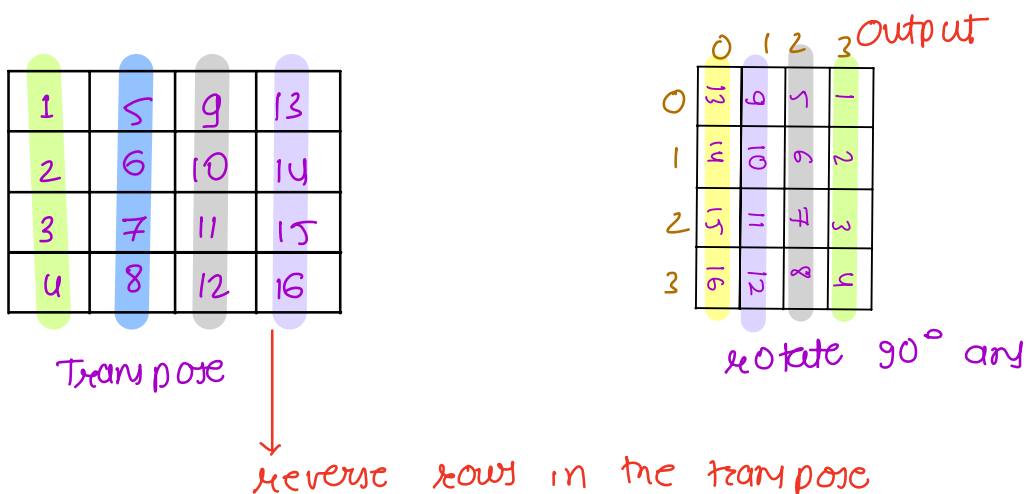


NOTE: Do not use extra space

Observation

0^{th} row $\longrightarrow$ 3rd col
 1^{st} row $\longrightarrow$ 2nd col
 2^{nd} row $\longrightarrow$ 1st col
 3^{rd} row $\longrightarrow$ 0th col

$\therefore$ row $\longrightarrow$ col . Let's try transpose



13	9	5	1
14	10	6	2
15	11	7	3
16	12	8	4

Transpose

	0	1	2	3	Output
0	13	9	5	1	
1	14	10	6	2	
2	15	11	7	3	
3	16	12	8	4	

rotate 90° anti

Overall Idea

- 1> Transpose given matrix
- 2> Reverse every row in the transposed matrix.

Pseudocode

`transpose(A)` $\nearrow O(N^2)$
 for $k \rightarrow 0$ to $N-1$ { $O(N^2)$
 `reverse(A[k])` }
 }

TC: $O(N^2)$
 SC: $O(1)$

Doubt Session